2024 IEEE 25th International Conference on High Performance Switching and Routing (HPSR) | 979-8-3503-6385-2/24/$31.00 ©2024 IEEE | DOI: 10.1109/HPSR62440.2024.10635926

Generating P4 Dataplanes Using LLMs

Mihai-Valentin Dumitru, Vlad-Andrei Bădoiu, Alexandru M. Gherghescu, Costin Raiciu
University Politehnica of Bucharest
{mihai.dumitru2201, vlad_andrei.badoiu, agherghescu2411, costin.raiciu}@upb.ro

***Abstract*—Large Language Models (LLMs) have recently become the source of impressive results in code generation, but there are still areas left neglected. Virtually all small and medium-sized LLMs are unable to produce code in P4, the most popular language for programmable dataplanes.**

Automatically generating dataplane code carries the promise of flexible networks that can quickly adapt to their specific conditions at the lowest level. P4 is structurally simpler than general-purpose languages, but also offers a much smaller corpus of existing programs, thus setting up interesting challenges for deep-learning based code generation. In this paper, we set out to investigate the extent to which current State-of-the-Art LLMs can produce P4 code and we push these boundaries by experimenting with finetuning to produce code generators that are open, robust and use low resources (1B-3B parameters). To evaluate the quality of the generated code, we have developed a novel benchmark for P4 auto-completion tasks. Our results show that fine-tuned models using the specialized dataset outperform ChatGPT 4 and Gemini Ultra, both in terms of compilability and alignment.

***Index Terms*—P4, programmable dataplanes, LLM, code generation**

I. Introduction

The ongoing advancements in the field of LLMs have revolutionized how we approach various software engineering tasks, significantly enhancing productivity and innovation in this sector. LLMs, powered by their ability to understand and generate natural language, are now instrumental in a wide range of applications within software engineering [1], including but not limited to requirements engineering, software design, and software development.

In practice, we find that LLMs perform exceptionally well in generating high-quality code for programming languages that are prevalent in their training datasets. However, their performance significantly declines when dealing with less-represented languages specific to niche domains such as dataplane programming. For instance, P4 is absent from major coding datasets like The Stack [2]. Even when such languages are included, they often make up only a small, potentially uncleaned fraction of the data, as in The Stack v2 [3].

In this context, recent studies have shown that the performance of LLMs for code generation is significantly influenced by the quality [4] and characteristics of the dataset, as well as the complexity of the language involved [5]. For instance, the study by Jain et al. [4] highlights how a model trained on a smaller amount of higher-quality data can outperform a model trained on the original larger, non-cleaned dataset. Another notable example is the phi-1.5 model [6], trained exclusively on a textbook dataset, that surpasses in performance models three times its size trained on much larger datasets. These findings underscore the importance of dataset quality over quantity and give us hope that LLMs could be used in domains where they employ domain specific languages with only a few megabytes of available code samples.

We theorize that dataplane programming in P4 exhibits the necessary characteristics to enable models to learn it efficiently given a specialized dataset. The language is simple in structure and similar to C, meaning that we can transfer C syntax knowledge. There are no loops and the usual P4 code is size limited by the available memory on the switch. Moreover, due to its non-Turing-completeness, we can leverage verification frameworks [7]–[11] for P4 to generate higher quality code, without bugs, by connecting their output to a prompt engineering framework. With this work, we pave the way to generate P4 from natural language or specialized text such as RFCs for easier networking prototyping, research and development.

In this paper, we develop a specialized dataset for P4-16, by curating code available online and enhancing it with targeted insights. This includes discussions related to P4, networking knowledge in the form of Request For Comments (RFCs), and technical papers to bridge language and networking knowledge. Besides employing state-of-the-art code cleaning techniques [2], [4], we synthetically introduce code comments to better connect natural language descriptions and text implementations. Our work outlines the methodology employed in assembling such a dataset amidst the scarcity of relevant data on the internet. We provide an analysis on how various dataset parameters impact the model's performance, providing a comprehensive understanding of the dataset's influence on the performance of the model.

For our evaluation, we employed finetuning on small (1-3 billion parameters) open models for code generation in P4. To assess the performance of the trained models, we developed a benchmarking methodology that evaluates a model's effectiveness in dataplane programming, even if it does not produce syntactically correct programs. Our results demonstrate that finetuning with a specialized P4 dataset is sufficient to generate code that rivals the performance of OpenAI's GPT-4 in P4 code generation, even with far smaller models.

We start by describing P4 and the challenge it presents for LLM code generation in §II, then we proceed to analyze the ability of existing state-of-the-art models to generate P4 code. Motivated by the low-quality of current models, we set out to create our own in §IV: we design a specialized dataset and use it to finetune several open models. §V describes the implementation of a set of benchmarks for evaluating our model, together with an analysis of its performance, using

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on August 23,2026 at 17:10:05 UTC from IEEE Xplore. Restrictions apply.

existing models as a baseline. We discuss opportunities for future work in §VI and present our conclusions in §VII.

II. Background

P4 [12] is an open and widely-used language for programmable dataplanes. It has a C-like syntax, but with much simpler structure. Notably, there are no loops, no jumps, no recursion, no pointers or memory management. P4 dataplanes consist of a number of programmable blocks, such as ingress and egress control blocks, parsers, deparsers, checksum calculators. The structure and syntactical elements of a control block differ significantly from those of a parser, thus P4 can be thought of as consisting of several specialized sub-languages.

The original version of the language, P4-14, was soon superseded by P4-16, which abstracted away the switch architecture from the language itself. Architectures specify the configuration and interactions of programmable and non-programmable blocks, mechanisms for forwarding, recirculating and dropping packets, available metadata etc. The standard language itself is not sufficient for development; a P4 program must be written for a particular architecture. In this paper, we will focus on the `v1model` architecture, due to its popularity.

These features make P4 a promising challenge for code generation. The problem of automatically generating P4 code has been tackled before [13]–[20]. However, to our knowledge, there has been no prior effort to use LLMs for this purpose.

Numerous works employ "classical" methods for generating P4 code. Graph-to-P4 [13] can convert a graph of headers into the code for a P4 parser. Motivated by the idea of moving computation at the edge of the network, P4rrot [14] provides a language for specifying application-level processors which are then automatically translated into P4 in a classical rule-based manner. Lucid [15] allows the programmer to employ powerful abstractions such as "events" and "handlers" for packet processing that help avoid bugs and hide away dataplane details. Lucid 2.0 [16] builds on top of this by adding new constructs to achieve *pipeline-safety*. Lyra [17] provides a "*one-big-pipeline*" abstraction to the programmer, allowing them to specify their intent in "simple statements" that then get converted into dataplane code for one or more actual devices. Homunculus [18] is a framework that can transform declarative requirements for Machine Learning applications into a concrete implementation for the switching hardware, which may include P4 dataplanes.

GP4P4 [19] uses *genetic programming* to create P4 dataplanes starting from "behavioral rules": a set of preconditions and postconditions on the header fields and metadata of the processed packets. A genetic programming approach is also presented in [20], which also employs *federated learning* for a decentralized cross-domain collaboration in acquiring data and learning from it.

Focusing on machine learning-based code generation, many open LLMs address the task of code generation [3], [21]–[26]. These models are usually trained on openly available code datasets [2], [3], [27], which often do not feature P4 samples. One exception is the newly released The Stack v2 [3], the largest publicly available code dataset at this time, which contains samples from over 600 programming languages; but P4 is severely underrepresented, with only 70MB of code. The authors of The Stack v2 refer to Julia and Perl as "low-resource languages", with 6.12 and 7.82 GB of code available.

III. Do existing models know P4?

In order to compare existing models and assess the effectiveness of various improvement strategies, we need a quality metric for generated code. At the time of writing, even the state-of-the-art models that can generate P4 were unable to consistently output syntactically correct programs. This makes functional evaluation in the style of HumanEval [28] difficult, as we cannot run tests on programs which don't compile.

A. Evaluation methodology

We have created two task categories to evaluate P4 generation capabilities, focusing on code completion and the specific intricacies of dataplane programming:

Code generation for tasks. We developed cleaned-up, slightly modified versions of the problem statement for the following P4 tutorials[1]: basic forward, basic tunneling, ECN, MRI, source routing, calculator, load balancing, QOS, firewall and link monitor. Our changes consists of adding new tasks, changing the code structure and updating the task instruction. For each tutorial, we have a version including the starting code and one without.

The P4 tutorials, together with the reference solutions are very likely part of models' training sets. Our changes aim to avoid potential "copy-paste" solutions; but even after the changes, the resulting tasks are similar to the original ones; thus correct solutions alone are not good indicators of generative capabilities. But failure of solving this task is a good indicator of the model's low capabilities in producing and understanding P4 code.

Auto-completion tasks. These are 15 snippets of P4 code, expecting the implementation of a particular syntactical element, such as a control block, a table, a parser state, an action etc. Some of these are accompanied by descriptive comments of the desired functionality, while others rely simply on names or surrounding context. The tests include tasks such as:

- given some new header `foo` and macro definitions of the form `IP_PROTO_FOO`, as well as a partial parser that deals with Ethernet, followed by IPv4, the model is expected to complete the `parse_ipv4` state.
- given some new header definitions and a parser implementation, the model is expected to generate the deparser
- given the name and comment-description of an action, the model is expected to complete the action body

To motivate our work we use the code generation tasks to survey existing models' capabilities regarding P4. A summary of the results is presented in Table I, where the slash is used to separate the results between code generation with and without a skeleton. Nearly all open models are not able to write

[1] https://github.com/p4lang/tutorials

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on August 23,2026 at 17:10:05 UTC from IEEE Xplore. Restrictions apply.

P4 at all, except for Starcoder which performs poorly. Only proprietary models ChatGPT 4 and Gemini are promising as ready-to-use tools.

Model Name	# Programs Compile	# Tests Pass	# Programs compile after feedback
GPT4 Turbo	9/1	8/0	1/1
GPT3	1/0	0/0	0/0
StarCoder 2 (15B)	0/0	0/0	0/0
Gemini	0/0	0/0	0/0
Magicoder DeepSeek-Coder, OLMo, Code LLama, Gemma	0/0	0/0	0/0

TABLE I
OVERVIEW OF MODELS PERFORMANCE ON P4 CODE GENERATION TASKS.

B. ChatGPT

ChatGPT 4 is the best performing model in our benchmark. Given the skeleton of the 9 tutorial tasks, it managed to produce code that compiles in 6 cases. By providing the compiler's output in the feedback loop it was able to produce compilable code for one extra task. Two more solutions need only one line of code change: removing a `for` loop over the elements of a header stack in the deparser. Eight out of these nine solutions pass all unit tests relevant to the task.

The tutorial that compiles but fails the tests is the `mri` exercise. A newly added header is not set valid; the official solution contains a `setValid()` statement preceded by a commentary explaining how this is explicitly needed in P4-16, as opposed to P4-14. Providing ChatGPT with this comment helps it to generate the fix and pass the tests. Overall, we noticed the following common mistakes done by ChatGPT 4:

- header validity checks in the deparser; this is unnecessary and, although not explicitly prohibited by the P4-16 standard, it is not supported on BMv2
- C-style `for` loops in the deparser to emit the elements of a header stack; the correct way to do this is to `emit` the entire header stack
- no explicit casts, between bit fields of different width
- wrong header stack syntax: `type var[10];` instead of `type[10] var;`

Other mistakes include: trying to `typedef` an anonymous structure, using character constants and declaring new variables using `var`; none of these are P4 constructs.

The syntactic quality of the generated code dramatically decreases when we remove the skeleton and leave only the task statement: just one solution compiles. Feedback from the compiler doesn't help: for four tasks, it regenerates the exact same error; for three others it misunderstands what the problem is.

One of the most frequent mistakes is placing tables and actions outside the scope of a control block: this is valid in P4-14, but not in P4-16. It was not clear from our experiments with ChatGPT 4 whether this issue stems from the presence of P4-14 programs in the training data; but ChatGPT 3.5 seems prone to utilize the P4-14 syntax in other contexts, such as writing parsers and header definitions.

ChatGPT also shows a lack of familiarity with some target-specific constructs. Tasked to produce code for the `v1model` architecture, it correctly generates the necessary blocks and instantiates a 'V1Switch' with the proper parameters. However, it misspells `Register` instead of `register`, doesn't use `hash` but a hypothetical `hash5`, explicitly stating that the concrete hash function depends on the target.

ChatGPT 3.5 generates code with far more errors; even given the skeleton, no solution compiles and it is not able to improve from compiler feedback. It uses P4-14 syntax for the parser much more often; as well as placing tables and actions outside the scope of a control block. At times, it hallucinates headers and data types; one of these being `time_t`, again suggesting that knowledge of other programming language may hinder this task. Even though explicitly tasked to provide a complete program P4 program, both versions occasionally fail to do so, instead leaving sections marked as "`TODO`".

C. Gemini

Google's Gemini [29] does not yield promising results, but the Ultra version seems at least superficially on par with ChatGPT 4. The web client has an output bug that omits the angle bracket notation that specifies the size of bit strings, but we assume that these are always correct.

Much like ChatGPT, Gemini mixes in P4-14 syntax for headers and parser, and sometimes places actions and tables outside the scope of a control block. Other shared problems include: validity checks in the deparser, declaring variables with the keyword `var`, not including the proper headers, wrong usage of platform primitives such as hashes and registers.

Much more often, it does not output a complete program even when specifically required to do so, but produces a verbose output with various interspersed snippets. Some mistakes seem to be of a *conceptual* nature, rather than purely syntactical. For example, in several instances, the IPv4 checksum is recalculated in the forwarding action; the TTL is sometimes not decremented.

D. Open models

Among the open models we tried, only StarCoder 2 15B is trained on a dataset that contains P4 [3] and is able to produce P4. Unlike ChatGPT, StarCoder is not designed to answer instructions and so performs poorly on the tutorial tasks, sometimes extending the text, instead of writing a dataplane. It can produce syntactically correct code, but in many cases it just seems to replicate verbatim code from its training set.

Other Stare-of-the-Art code generators, such as Magicoder [30], DeepSeek-Coder [31] (7B), OLMo [21], Code LLama [26], Gemma [25] (7B and 13B) are not able to produce satisfactory code due to P4 missing entirely from their training dataset.

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on August 23,2026 at 17:10:05 UTC from IEEE Xplore. Restrictions apply.

IV. Building our own model

A. Designing a Specialized Dataset

One of the main ways to improve open models is finetuning. To this end, we designed a specialized dataset for generating P4 code from natural language; it includes both code, relevant knowledge on networking and a glue between the two. Our dataset consists of 10746 documents, an overview of the split can be consulted in Table II.

Category	Doc Type	# of docs	Size(MB)
P4 sources	Code	9477	46.6
Discussion threads	Code & Text	381	1.3
Scientific articles	Code & Text	368	14
RFCs	Text	519	25.2
P4-16 standard	Text	1	0.2
Total	Code & Text	10746	87

TABLE II
Overview of the P4 corpus.

We started by gathering all the P4 repositories available on GitHub. We further include sources from the Software Heritage project [2] via The Stack v2 [3], obtaining 44010 files, totalling 202MB; we subject these to a commonly used cleaning pipeline for code [23].

Removing P4-14. A complicating factor for our task of generating P4 code is that "P4" actually refers to two different languages. P4-14, the original version of the language, first released in 2014 and P4-16 – a replacement that followed just two years after. The two languages differ drastically, both in overall structure and in specific syntactic elements, such as headers, parsers and control blocks. We remove all P4-14 files (6055 documents, totalling 44 MB), filtering them by employing regex-based searches for syntactic constructs that don't appear in P4-16. Based on our observation in §III, about how models like ChatGPT and Gemini often mix in P4-14 syntax, we believe this to be an important step.

As mentioned in §II, P4 programs are written for particular architectures. We choose to focus on inference for the `v1model` architecture, but we keep code written for other architectures in the dataset. This is because we want as much data as possible and our results with existing LLMs don't indicate that mixing architecture constructs is an issue.

Filtering non-P4 files. We further filter from the dataset mislabeled files which don't contain P4 code; we do this manually, through heuristic regexes. We remove several files which were marked as having been auto-generated and one which was created by the `p4obfuscator`. We also remove empty files and those larger than 500KB. One of the main difficulties in creating the dataset is that quite a substantial number of P4 files appear to have at least part of them programmatically generated. We choose to keep most of these, only discarding the files which don't contain any other syntactic constructs besides a long list of lines differing only in a few characters/numbers. This filtering step discards 3316 documents, totalling 32 MBs.

Deduplication We employ MinHash [32] to filter near-duplicates with a 0.85 Jaccard similarity threshold. This removes 25162 documents, totalling 82 MBs.

Networking knowledge At this point we have the code part of the dataset. A successful code-generator should be able to map natural language instruction to the target programming language. To achieve this, we introduce the relevant knowledge and glue between knowledge and code in several ways. First, we extend the dataset with networking knowledge about protocols in the form of RFCs, from RFC 791 to 2000. We then add 381 entries from the P4 discussion forums[3]. We add around 350 P4 articles, collected from an extensive survey [33] as well as the May 2023 revision of the P4-16 standard.

Adding code comments. One of our aims is for our model to be usable for auxiliary tasks including adding code comments and refactoring. We leverage a teacher LLM to enhance the code portion of our dataset, by synthetically introducing new comments, summarizing existing functionality. We use ChatGPT 3.5 due to its abundance of domain knowledge, fast response time and similar quality of comments with GPT 4 at this task. We picked source files with lower than 10 comments and added comments to 18% of the source files.

B. Finetuning existing models

We finetuned several models, all with a context length of 2048: the 1B version of StarCoder [23], the 2B version of Gemma [25] and the 3B version of StarCoder 2 [3]. We did not finetune larger models because of limited resources.

For finetuning, we used qLoRA [34], with an *alpha* value of 32 and an *r* value of 8, with a learning rate of 10^{-4}, to encourage slow domain-adaptation and prevent catastrophic forgetting, and a weight decay of 0.01. The batch size was kept constant at 32, and the learning rate was annealed by a cosine schedule, with an initial warmup of 5% of the total number of steps. We ran the training for a single epoch for all models, as we feared overfitting might quickly become a problem, because the dataset contains a lot of repetitive boilerplate. The number of trainable parameters through qLoRA amounted to less than 0.5% of the model's total number of parameters, which allowed for short finetuning times: around four hours on a single Nvidia A100 40GB for the StarCoder 2 3B on the full dataset. In Figure 1 we plotted the evaluation loss on a small held-out part of the full dataset. There is no indication of overfitting.

V. Finetuned Model Evaluation

We ran all the finetuned models on the autocompletion benchmark described in §III. We use ChatGPT 4 and Gemini Ultra as baselines, prompting them to act as autocomplete tools. We evaluated only the completion capabilities of the models because our dataset does not target instruct models: they don't interpret text prompts as tasks that need to be solved, but as streams of text that need to be continued, which they sometimes do by emitting additional requirements, examples or diagnostics.

[2]https://www.softwareheritage.org/

[3]https://forum.p4.org/

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on August 23,2026 at 17:10:05 UTC from IEEE Xplore. Restrictions apply.

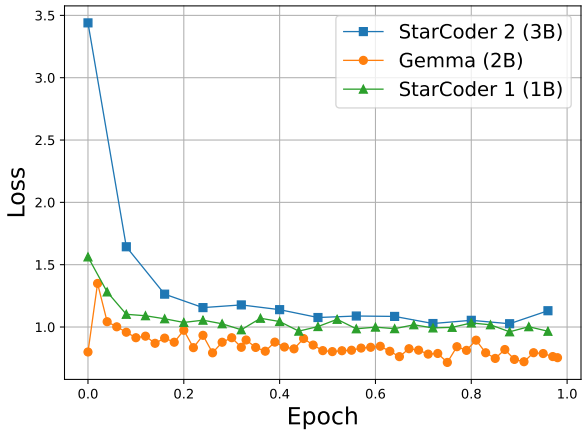


Fig. 1. Evaluation loss during the finetuning process.

Each task's reference solution consists of two to ten lines of code.

We perform manual postprocessing in which we trim the output, depending on context: if the model was supposed to autocomplete the definition of a parser state, we delete everything that follows that parser state, before assessing the correctness of the answer. We evaluate the following characteristics of the output code:

Does it compile? We take the postprocessed output of the model and insert it into a minimal, complete P4 program template, then run it through the `p4c` compiler. We ignore warnings and consider the task successful if there are no compilation errors.

Is it logically coherent? Compilation alone is not sufficient. In one case, the model kept generating a long list of variable declarations; while syntactically correct, these did not address the task at hand. This requirement is also failed by the presence of any syntax error, with the exception of size mismatches (using the wrong hash size, implicit casts, etc.). We allow these errors because they seem "light", in the sense that they could be easily fixed by additional tools.

Does it implement all functionality requested? We use this to distinguish proper solutions from *partial solutions* which, although coherent and free of compilation errors, do not contain all the code necessary to solve the task.

The results are summarized in Table III; For the purposes of an ablation study, our three base models were finetuned on three datasets: one that contains only filtered, deduplicated P4-16 code (`code`); one that also contains text documents in the form of RFCs, forum discussions, scientific articles and the P4 standard (`+text`) and another that contains both text and code with comments added to 18% of sources (`+comments`).

Compared to the finetuned models, ChatGPT 4 makes more severe syntactical errors, such as including `for` loops in the deparser. This supports the idea that dataset quality is a key part of a model's performance. Gemini generates syntax errors more often and sometimes more severe, such as omitting a closing brace for the parser block. For some tasks, it produced no relevant code.

Model	Dataset	Compiles?	Coherent?	All?
SC 1B	code	9	8	7
SC 1B	+text	11	8	8
SC 1B	+comments	12	9	6
Gemma 2B	code	9	10	9
Gemma 2B	+text	11	9	8
Gemma 2B	+comments	11	8	8
SC2 3B	code	12	10	8
SC2 3B	+text	10	10	8
SC2 3B	+comments	12	9	8
Gemini Ultra		5	4	4
ChatGPT 4		6	7	7

TABLE III
RESULTS OF THE FINETUNED MODELS ON THE 15 TASKS OF THE AUTOCOMPLETION BENCHMARK.

Five tasks were solved correctly by all models-under-test. These involved writing two simple actions to calculate the maximum and sum of their arguments respectively. The other three required filling in a table's action list, parsing a new type of header and writing a simple deparser.

The notable difference between models trained on our dataset and ChatGPT4 lies in the deparser: for 3 more tasks, our models produce a deparser that compiles; ChatGPT often messes up the syntax, employing loops and validity checks.

Other syntax errors performed by ChatGPT, such as declaring actions outside the scope of a control block, or using non-boolean expressions as conditions for an `if`, are not present in our models' output and are likely a result of having P4-14 sources mixed into the training set of ChatGPT.

Two of the tasks were "trick questions", asking the model to produce the body of actions that compute the GCD of two numbers using Euclid's algorithm or determine if a number is prime. All of the models tested, produce C-like implementations employing either a `while` loop, or recursion. Neither control loops, nor recursion, are valid P4 constructs. Both StarCoder 1 and StarCoder 2 sometimes produce solutions for primality testing which consist of an `if-else` chain that compares the number with the first primes in order (the output is cutoff at the context length).

Enhancing the dataset with text sources and synthetic comments does not have a noticeable effect.

VI. DISCUSSION AND FUTURE WORK

In §IV we presented our experiments of finetuning small, open models on a curated dataset. An obvious next step is to experiment with larger models and a larger context window.

We also plan to train an "instruct" [35] version of our model, which could generate P4 code based on natural language descriptions of the desired functionality. Evaluation of this model would require an additional benchmark suite.

The results in §V indicate that text resources and synthetic comments added to the dataset do not yield noticeable improvements in model performance. However, the benchmark tasks are focused on code-completion; an "instruct" model would allow us to explore natural language tasks, for which these dataset enhancements might prove useful.

Automatic code generation is a stepping stone towards developing robust, adaptable networks, capable of rewriting

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on August 23,2026 at 17:10:05 UTC from IEEE Xplore. Restrictions apply.

themselves at the lowest level, similar to the ideas presented in [18], [19], [36]. The code produced should not only be syntactically correct, but also efficient and secure. There is already a substantial amount of literature on employing P4 as the low-level backend for languages that operate with complex abstractions over a single switch or over an entire network [13]–[17]. Instead of producing raw P4 code, a model could be trained to generate such higher-level languages.

VII. Conclusions

In the world of LLMs, many code-generating models are unable to produce P4 code in any capacity. Only proprietary models show promise in this direction. However, their output is oftentimes riddled with syntactical errors and cannot be used as-is, without human intervention.

The closed nature of these models and the datasets they were trained on, as well as their intimidating size and underwhelming results, makes it desirable to explore the capabilities of smaller, open-source models. However, P4 is a severely underrepresented language in open datasets; it is absent from The Stack [2] and the low-amount present in The Stack v2 [3] is of low-quality, needing additional sanitization and a separation between P4-14 and P4-16 sources. In this paper we take a methodical approach to building a highly specialized P4 dataset and explore the aspects of training a model in the art of dataplane programming. Our experiments with fine-tuning on a highly curated dataset yield models whose code-completion output outperforms existing models using drastically less resources: between 1B and 2B parameters.

VIII. Acknowledgements

This work was partly funded by a VMWare research gift.

References

[1] X. Hou *et al.*, "Large language models for software engineering: A systematic literature review," *arXiv preprint arXiv:2308.10620*, 2023.

[2] D. Kocetkov *et al.*, "The stack: 3 tb of permissively licensed source code," *arXiv preprint arXiv:2211.15533*, 2022.

[3] A. Lozhkov *et al.*, "Starcoder 2 and the stack v2: The next generation," 2024.

[4] N. Jain, T. Zhang, W.-L. Chiang, J. E. Gonzalez, K. Sen, and I. Stoica, "Llm-assisted code cleaning for training accurate code generators," *arXiv preprint arXiv:2311.14904*, 2023.

[5] R. Eldan and Y. Li, "Tinystories: How small can language models be and still speak coherent english?," *arXiv preprint arXiv:2305.07759*, 2023.

[6] Y. Li, S. Bubeck, R. Eldan, A. Del Giorno, S. Gunasekar, and Y. T. Lee, "Textbooks are all you need ii: phi-1.5 technical report," *arXiv preprint arXiv:2309.05463*, 2023.

[7] J. Liu, W. Hallahan, C. Schlesinger, M. Sharif, J. Lee, R. Soulé, H. Wang, C. Caşcaval, N. McKeown, and N. Foster, "P4v: Practical verification for programmable data planes," in *Proceedings of the 2018 Conference of the ACM Special Interest Group on data communication*, pp. 490–503, 2018.

[8] A. Nötzli, J. Khan, A. Fingerhut, C. Barrett, and P. Athanas, "P4pktgen: Automated test case generation for p4 programs," in *Proceedings of the Symposium on SDN Research*, SOSR '18, (New York, NY, USA), pp. 5:1–5:7, ACM, 2018.

[9] M. Neves, L. Freire, A. Schaeffer-Filho, and M. Barcellos, "Verification of p4 programs in feasible time using assertions," in *Proceedings of the 14th ACM CoNEXT*, (New York, NY, USA), pp. 73–85, ACM, 2018.

[10] D. Dumitrescu, R. Stoenescu, L. Negreanu, and C. Raiciu, "bf4: towards bug-free p4 programs," in *Proceedings of the 2020 ACM SIGCOMM*, pp. 571–585, 2020.

[11] R. Stoenescu, D. Dumitrescu, M. Popovici, L. Negreanu, and C. Raiciu, "Debugging p4 programs with vera," in *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, pp. 518–532, 2018.

[12] P. Bosshart *et al.*, "P4: Programming protocol-independent packet processors," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, 2014.

[13] E. O. Zaballa and Z. Zhou, "Graph-to-p4: A p4 boilerplate code generator for parse graphs," in *2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*, pp. 1–2, IEEE, 2019.

[14] C. Györgyi, S. Laki, and S. Schmid, "P4rrot: Generating p4 code for the application layer," *ACM SIGCOMM Computer Communication Review*, vol. 53, no. 1, pp. 30–37, 2023.

[15] J. Sonchack, D. Loehr, J. Rexford, and D. Walker, "Lucid: A language for control in the data plane," in *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, pp. 731–747, 2021.

[16] D. Loehr and D. Walker, "Safe, modular packet pipeline programming," *Proceedings of the ACM on Programming Languages*, vol. 6, no. POPL, pp. 1–28, 2022.

[17] J. Gao *et al.*, "Lyra: A cross-platform language and compiler for data plane programming on heterogeneous asics," in *Proceedings of the 2020 ACM SIGCOMM*, pp. 435–450, 2020.

[18] T. Swamy, A. Zulfiqar, L. Nardi, M. Shahbaz, and K. Olukotun, "Homunculus: Auto-generating efficient data-plane ml pipelines for datacenter networks," in *Proceedings of the 28th ACM ASPLOS, Volume 3*, pp. 329–342, 2023.

[19] M. Riftadi, J. Oostenbrink, and F. Kuipers, "Gp4p4: Enabling self-programming networks," *arXiv preprint arXiv:1910.00967*, 2019.

[20] O. Hireche, C. Benzaïd, and T. Taleb, "Deep data plane programming and ai for zero-trust self-driven networking in beyond 5g," *Computer Networks*, vol. 203, p. 108668, 2022.

[21] D. Groeneveld *et al.*, "Olmo: Accelerating the science of language models," *arXiv preprint arXiv:2402.00838*, 2024.

[22] H. Touvron *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[23] R. Li *et al.*, "Starcoder: may the source be with you!," *arXiv preprint arXiv:2305.06161*, 2023.

[24] E. Almazrouei *et al.*, "The falcon series of open language models," *arXiv preprint arXiv:2311.16867*, 2023.

[25] G. Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love, *et al.*, "Gemma: Open models based on gemini research and technology," *arXiv preprint arXiv:2403.08295*, 2024.

[26] B. Rozière *et al.*, "Code llama: Open foundation models for code," 2024.

[27] L. Gao *et al.*, "The pile: An 800gb dataset of diverse text for language modeling," *arXiv preprint arXiv:2101.00027*, 2020.

[28] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[29] G. Team *et al.*, "Gemini: A family of highly capable multimodal models," 2023.

[30] Y. Wei, Z. Wang, J. Liu, Y. Ding, and L. Zhang, "Magicoder: Source code is all you need," *arXiv preprint arXiv:2312.02120*, 2023.

[31] D. Guo *et al.*, "Deepseek-coder: When the large language model meets programming – the rise of code intelligence," 2024.

[32] A. Z. Broder, "Identifying and filtering near-duplicate documents," in *Annual symposium on combinatorial pattern matching*, pp. 1–10, Springer, 2000.

[33] F. Hauser *et al.*, "A survey on data plane programming with p4: Fundamentals, advances, and applied research," *Journal of Network and Computer Applications*, vol. 212, p. 103561, 2023.

[34] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized llms," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[35] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," *Advances in neural information processing systems*, vol. 35, pp. 27730–27744, 2022.

[36] L. Yu, J. Sonchack, and V. Liu, "Mantis: Reactive programmable switches," in *Proceedings of the 2020 ACM SIGCOMM*, pp. 296–309, 2020.

Authorized licensed use limited to: XIDIAN UNIVERSITY. Downloaded on August 23,2026 at 17:10:05 UTC from IEEE Xplore. Restrictions apply.